Data Engineering & Analysis Report

REFERENCE FILE: AIR_INDIA_SQL_PROJECT_ADVANCED.SQL

Executive Summary

This report documents the architectural design and analytical capabilities of the advanced Air India database project. The project establishes a robust relational database schema to manage airline operations, tracking flights, routes, aircraft, and passenger bookings. A defining characteristic of this project is its strict adherence to raw, optimized SQL queries—intentionally bypassing Common Table Expressions (CTEs), views, and subqueries to demonstrate advanced proficiency in direct table joins, aggregate filtering, and window functions.

Database Schema Architecture

The relational database (`Air_india_db`) is normalized to ensure data integrity across operational and transactional entities. The schema is built upon seven foundational tables linked via primary and foreign key constraints.

Core Entities

- **airlines:** Global airline metadata
- **airports:** Location & IATA codes
- **aircraft:** Fleet models & capacity

Operations

- **routes:** Origin/Destination distances
- **flights:** Scheduled departures & status

Transactions

- **passengers:** Demographic details
- **bookings:** Seat class, fares & status

Strict Design Constraints: All 32 analytical queries executed in this project are strictly limited to base table joins (INNER, LEFT, SELF), aggregation clauses (GROUP BY, HAVING), and advanced Window Functions. Subqueries and user-defined functions were explicitly excluded to validate core SQL proficiency.

Core Analytical Capabilities

The queries engineered for this project extract high-level business intelligence directly from raw transactional data. The analysis is divided into progressive logical sections, moving from basic data extraction to complex row-level comparative analytics.

A. Foundational Analysis (WHERE/JOIN)

Utilizes complex multi-table joins to filter confirmed high-value bookings. It includes self-joins on the `routes` table to audit outbound versus return distances, and identifies data-quality anomalies (e.g., bookings created post-departure).

B. Aggregate Business Intelligence

Leverages `GROUP BY` and `HAVING` to generate structural insights. Key metrics include cancellation rates per airline, domestic versus international revenue segmentation, and monthly booking growth trends.

C. Row-Level Window Functions

Applies `RANK`, `LEAD`, and `LAG` to sequence passenger booking histories. It tracks fare drift across flights by comparing the first fare sold against subsequent bookings using `FIRST_VALUE`.

D. Advanced Hybrid Queries

Combines window functions over grouped data to compute advanced time-series metrics, such as 3-day moving averages of daily revenue and month-over-month growth percentages.

Key Business Questions Answered

- **Revenue Concentration:** Which flight routes contribute the highest percentage of total revenue within each airline?
- **Fleet Utilization:** What is the load factor (seats sold versus total capacity) for every scheduled flight, ranked by airline?
- **Customer Loyalty:** Categorizing passengers into Platinum, Gold, Silver, and Bronze tiers based on total historical spend using `NTILE(4)`.
- **Operational Audits:** Flagging schedule overlaps or position mismatches for specific aircraft by analyzing sequential flight paths via `LAG()`.

Technical Highlight: Pareto Analysis (80/20 Rule)

One of the most complex queries in the project (Q32) calculates the cumulative revenue share of passengers to identify the "Long tail" versus the top spenders. Achieving this without subqueries or CTEs requires chaining multiple window functions inside mathematical operations and `CASE` statements.

```
SELECT CONCAT(p.first_name, ' ', p.last_name) AS passenger_name, ROUND(SUM(b.fare), 2)
AS total_spend, ROUND(SUM(SUM(b.fare)) OVER (ORDER BY SUM(b.fare) DESC, p.passenger_id)
* 100 / SUM(SUM(b.fare)) OVER (), 1) AS cumulative_pct, CASE WHEN (SUM(SUM(b.fare))
OVER (ORDER BY SUM(b.fare) DESC, p.passenger_id) - SUM(b.fare)) * 100 /
SUM(SUM(b.fare)) OVER () < 80 THEN 'Top 80% revenue' ELSE 'Long tail' END AS
pareto_group FROM bookings b JOIN passengers p ON b.passenger_id = p.passenger_id WHERE
b.status = 'Confirmed' GROUP BY p.passenger_id, p.first_name, p.last_name ORDER BY
total_spend DESC;
```

This query demonstrates a deep understanding of logical processing order in SQL. By grouping by passenger first, the window function `SUM(SUM(b.fare)) OVER (...)` acts upon the aggregated rows to compute a running total, which is then divided by the grand total to establish the Pareto threshold dynamically.

Conclusion

The "Air India SQL Project (Advanced Edition)" successfully demonstrates how rigorous, standard SQL can solve complex business requirements without relying on intermediate tables or programmatic loops. The precise application of window functions alongside standard relational algebra provides a highly performant and scalable foundation for airline data analytics.